MACHINE LEARNING · LEVEL 6 · WEEK 4

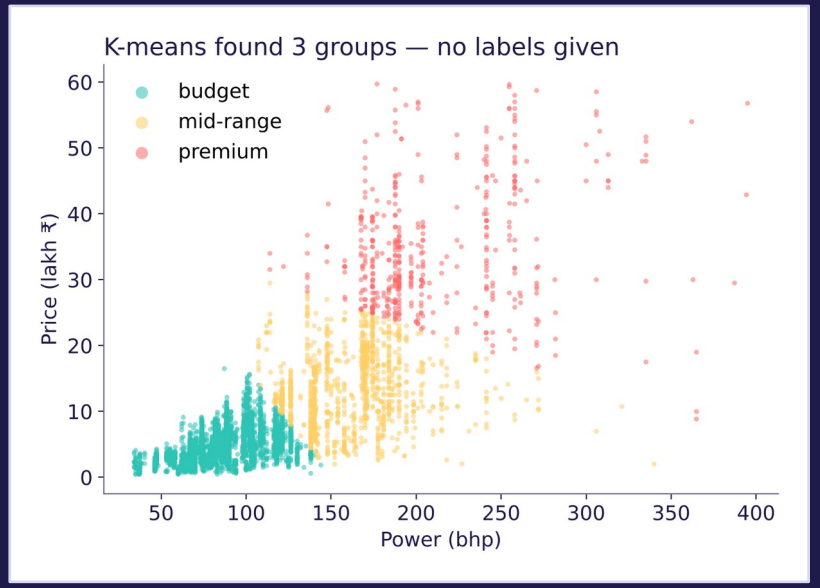
Machine learning and its frameworks

From examples to evaluation

What ML really is, the frameworks that build it, the pipeline that ships it, and the numbers that tell us whether it works.

Dr. Geetha Mohan · Future Education University College





BY THE END OF THIS WEEK

You will be able to…

1. Explain machine learning with everyday examples, and name features, labels, datasets and models
2. Tell supervised, unsupervised and reinforcement learning apart
3. Compare TensorFlow, PyTorch and fastai, and walk through the full ML pipeline
4. Build a confusion matrix and calculate accuracy, precision, recall, F1, specificity, MAE, MSE and RMSE



Today's regression model: predicted vs actual used-car prices

1

PART ONE · FOUNDATIONS

What machine learning actually means

Learning from examples instead of following hand-written rules.

Concepts
examples, vocabulary and the three types of learning

THE BIG SHIFT

Writing rules vs learning from examples

Traditional programming

A human writes the rules. Rules + data → answers. Example: IF the email contains 'lottery' THEN mark it as spam. It breaks as soon as spammers write 'l0ttery'.

Machine learning

A human supplies examples with the right answers. Data + answers → rules (a model). Show it 10,000 emails labelled spam or not spam, and it learns the patterns itself.

Why it matters

When the world changes, you retrain on new examples instead of rewriting thousands of rules by hand. The model keeps up with new spam tricks.

Definition Machine learning is a branch of AI where computers learn patterns from data to make predictions or decisions, without being explicitly programmed for every case.

YOU ALREADY USE ML

Machine learning in a normal day



Recommendations

Netflix, YouTube and Spotify learn from what you watched, skipped and replayed, and suggest what you will probably like next.



Spam filter

Your inbox has learned from millions of emails people marked as spam, so most junk never reaches you.



Maps and arrival time

Navigation apps learn from past traffic at that hour, on that road, to predict when you will arrive.



Face unlock

Your phone learned what your face looks like from a few scans, and checks it against the camera image every time.

Ask the room Which of these have you used today? What data did it need to learn from?

AN ANALOGY

How you learned to pick a ripe mango

1

Examples

As a child you saw many mangoes, and someone told you 'ripe' or 'not ripe'

2

Features

You noticed what mattered: colour, softness, smell

3

Pattern

Your brain built a rule nobody wrote down: yellow + soft + sweet smell = ripe

4

Prediction

Now you judge a mango you have never seen before

5

Feedback

A sour mango corrects you, so next time you get better

That is ML Examples → features → pattern (the model) → prediction on new data → improve with feedback.

KEY VOCABULARY

Dataset, features, label, model

Term	Meaning	Cars4U example
Dataset	The whole collection of examples	7,253 used-car listings in one CSV file
Sample / row	One example	One car: a 2015 Hyundai Creta in Pune
Feature (X)	An input the model uses to make a prediction	Year, kilometres driven, power, engine, fuel, transmission
Label / target (y)	The answer we want to predict	Price (in lakh ₹)
Model	The learned pattern: a function from features to label	Price ≈ 2.3 × automatic − 0.9 × age + …
Prediction (ŷ)	The model's answer for a new sample	A 2017 petrol Maruti Swift, 25,000 km → ₹6.9 lakh
Training	Adjusting the model so its predictions match the labels	Linear regression fitted on 4,674 cars

SEE IT IN THE DATA

Features go in, the label comes out

Name	Year	Km driven	Power	Transmission	Fuel	Price (label)
Maruti Wagon R LXI CNG	2010	72,000	58 bhp	Manual	CNG	₹1.75 L
Hyundai Creta 1.6 CRDi	2015	41,000	126 bhp	Manual	Diesel	₹12.50 L
Honda Jazz V	2011	46,000	88 bhp	Manual	Petrol	₹4.50 L
Audi A4 New 2.0 TDI	2013	40,670	141 bhp	Automatic	Diesel	₹17.74 L
New: Maruti Swift VXI	2017	25,000	82 bhp	Manual	Petrol	? → ₹6.9 L

Features (X) the six middle columns. **Label (y)** the price. The last row has no label: our model predicts ₹6.9 L (real 2017 Swifts sold for ₹5.3–6.2 L).

QUICK CHECK · 3 MIN

Feature or label?

1. Predicting whether a student passes: attendance %, quiz scores, pass/fail
2. Predicting house rent in Dubai: area, bedrooms, rent per year
3. Sorting fruit photos: pixel colours, fruit name
4. Detecting spam: words in the email, number of links, spam/not spam

SUGGESTED ANSWERS

Feature or label: suggested answers

1

Student pass/fail

Features: attendance %, quiz scores. Label: pass/fail (a classification task).

2

Dubai rent

Features: area, bedrooms. Label: rent per year (a regression task, because the answer is a number).

3

Fruit photos

Features: pixel colours. Label: fruit name. In images, the pixels are the features.

4

Spam

Features: words, number of links. Label: spam / not spam.

THREE WAYS TO LEARN

Supervised, unsupervised, reinforcement



Supervised

Learns from labelled examples: input → correct answer. Goal: predict the answer for new inputs. Examples: car price, spam filter, disease diagnosis, credit risk.



Unsupervised

No labels. Finds structure by itself: groups, patterns, unusual points. Examples: customer segments, grouping similar songs, fraud spotting, market basket analysis.



Reinforcement

Learns by trial and error. Actions earn rewards or penalties, and it maximises the total reward over time. Examples: game AI, robot walking, self-driving decisions.

Memory hook Supervised = a teacher with an answer key. Unsupervised = sorting your wardrobe with no instructions. Reinforcement = training a puppy with treats.

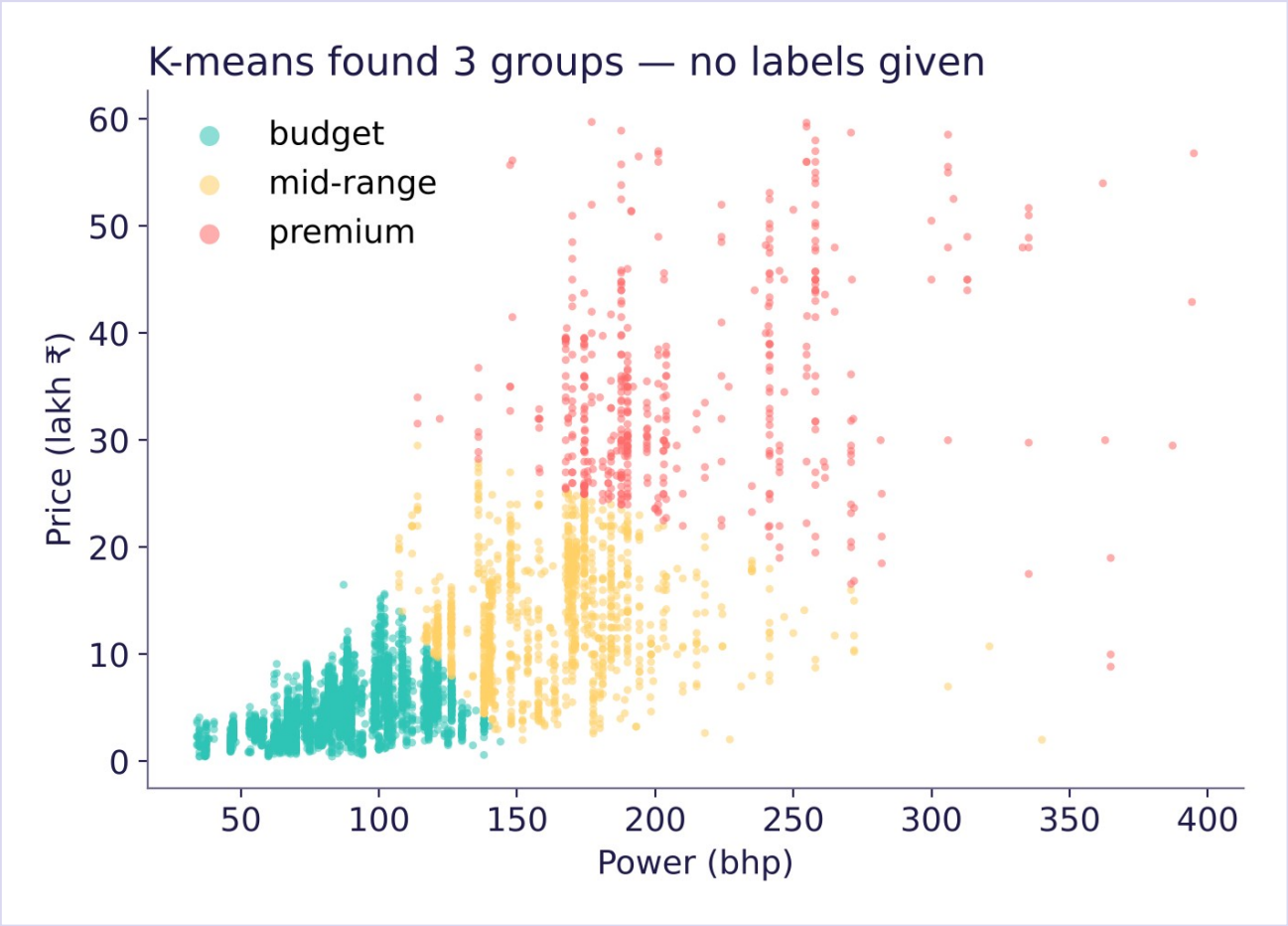
SUPERVISED · TWO FLAVOURS

Regression or classification?

	Regression	Classification
Output	A number on a continuous scale	A category (class) from a fixed list
Question it answers	How much? How many? How long?	Which one? Yes or no?
Cars4U example	What price will this car sell for? (₹7.3 lakh)	Will this car sell for more than ₹10 lakh? (yes / no)
Other examples	House rent, temperature tomorrow, delivery time	Spam filter, cat vs dog, sentiment (positive/negative/neutral)
How we score it	MAE, MSE, RMSE, R^2	Confusion matrix, accuracy, precision, recall, F1

UNSUPERVISED · EXAMPLE

Clustering cars with no labels



What happened here

- We gave K-means only two columns, power and price, and no answers
- We asked for 3 groups. It found budget, mid-range and premium by itself
- The names are ours: the algorithm only says group 0, 1, 2
- Business use: a dealer can price, stock and advertise each segment differently

REINFORCEMENT · HOW IT WORKS

Learning from rewards, one action at a time

1

Agent

The learner: a game player, robot or car

2

State

What it sees now: the board, the camera view

3

Action

It tries a move: left, right, jump, brake

4

Reward

+10 for a goal, −1 for crashing, 0 otherwise

5

Repeat

Over thousands of tries it learns which actions lead to the most reward

Example A robot vacuum: +1 for each clean tile, −5 for bumping the sofa. Nobody labels the 'right' move; it discovers a good strategy.

WHICH TYPE? · 4 MIN

Supervised, unsupervised or reinforcement?

1. Grouping 50,000 shoppers by buying habits for a marketing campaign
2. Predicting tomorrow's electricity demand from past years
3. Teaching a drone to land using rewards for soft landings
4. Flagging emails as phishing using 20,000 labelled examples
5. Finding unusual bank transactions with no fraud labels

SUGGESTED ANSWERS

Which type: suggested answers

1

Grouping shoppers

Unsupervised (clustering): no labels, we want groups.

2

Electricity demand

Supervised regression: past demand is the label, and we predict a number.

3

Drone landing

Reinforcement: learns from rewards, with no labelled 'correct' moves.

4

Phishing emails

Supervised classification: labelled phishing / safe examples.

5

Unusual transactions

Unsupervised (anomaly detection): no fraud labels, so look for what does not fit.

2

PART TWO · TOOLS

TensorFlow, PyTorch and fastai

Frameworks do the heavy maths so you can focus on the problem.

Frameworks
what they do, how they differ, the same model in each

WHAT IS A FRAMEWORK?

A professional kitchen, already set up



Tensors and maths

Fast multi-dimensional arrays (tensors) and thousands of ready-made maths operations.



Automatic gradients

Works out how to adjust every weight (backpropagation), so you never do the calculus by hand.



Ready-made layers

Dense, convolution and attention layers, loss functions and optimisers you just plug together.



GPU and deployment

Runs on graphics cards for speed and exports models to phones, browsers and servers.

Analogy You could build an oven from scratch before cooking, or walk into a kitchen where the oven, knives and pans are ready. A framework is the ready kitchen.

SIDE BY SIDE

Choosing between the three

	TensorFlow (+ Keras)	PyTorch	fastai
Made by	Google (2015)	Meta / Facebook (2016)	Jeremy Howard and Rachel Thomas (2018)
Style	High-level Keras API: model.fit()	Pythonic; you write the training loop	Very high level, built on top of PyTorch
Strengths	Production, mobile (TF Lite), web (TF.js)	Flexible, easy to debug, research favourite	Best practice by default; great results in a few lines
Best for	Deploying at scale in apps and products	Research, custom models, learning how it works	Beginners, fast prototypes, transfer learning
Lines for a basic model	≈ 6	≈ 10 (loop included)	≈ 4

Same ideas everywhere data → model → loss → optimiser → train → evaluate. Learn the ideas once and you can switch frameworks.

TENSORFLOW / KERAS

Predicting car prices with Keras

```
In [ ]:
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(1)              # one number: price
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
model.fit(X_train, y_train, epochs=50, validation_split=0.2)
model.evaluate(X_test, y_test)
```

Sequential stack layers in order
compile choose how to learn (adam) and what to minimise (MSE)
fit the training loop, hidden for you
evaluate score on unseen test data

Keras hides the loop: one fit() call does guess, measure, adjust, repeat.

PYTORCH

The same model, with the loop in view

```
In [ ]:
import torch, torch.nn as nn

model = nn.Sequential(nn.Linear(8, 32), nn.ReLU(), nn.Linear(32, 1))
loss_fn = nn.MSELoss()
opt = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(50):
    pred = model(X_train)              # 1 guess
    loss = loss_fn(pred, y_train)      # 2 measure
    opt.zero_grad(); loss.backward()   # 3 find downhill
    opt.step()                         # 4 nudge weights
```

8 inputs our 8 car features
The loop you write it, so you see every step
backward() automatic gradients
Why bother? full control for custom ideas

PyTorch shows you the four steps that every network shares: guess, measure, find downhill, nudge.

FASTAI

Tabular data in four lines

```
In [ ]:
from fastai.tabular.all import *

dls = TabularDataLoaders.from_df(df, y_names="Price",
        cat_names=["Transmission", "Fuel_Type"],
        cont_names=["Year", "power", "engine"],
        procs=[Categorify, FillMissing, Normalize])
learn = tabular_learner(dls, metrics=rmse)
learn.fit_one_cycle(5)
```

procs cleaning built in: encode categories, fill gaps, scale numbers
tabular_learner picks a sensible network for you
fit_one_cycle a proven training schedule
Built on PyTorch you can drop down a level when needed

fastai packs best practice into defaults, so beginners get strong results fast.

YOU DECIDE · 3 MIN

Which framework would you choose?

1. A startup wants a model inside its Android app by next month
2. A PhD student is inventing a new kind of neural network layer
3. A design student wants an image classifier for her portfolio this weekend
4. A bank needs a model that serves millions of requests a day

SUGGESTED ANSWERS

Which framework: suggested answers

1

Android app

TensorFlow: TF Lite is built for phones. (PyTorch Mobile / ExecuTorch also works.)

2

New layer for research

PyTorch: flexible, easy to debug, and what most research code uses.

3

Weekend image classifier

fastai: transfer learning in a few lines, with strong default settings.

4

Millions of requests

TensorFlow (TF Serving) is a classic choice; PyTorch with TorchServe is also used. Reliability matters more than the brand.

3

PART THREE · PROCESS

The complete machine learning pipeline

From a business question to a model that keeps working.

Pipeline
ten steps, then training, validation, testing, leakage and overfitting

THE PIPELINE · STEPS 1–5

From question to model-ready data

1

Define the problem

What do we predict, for whom, and how do we measure success? Cars4U: price, scored with RMSE.

2

Collect data

Gather examples with labels. Cars4U: 7,253 listings across 11 cities.

3

Clean

Fix types, duplicates, impossible values. '58.16 bhp' → 58.16; one car with 6.5 million km fixed.

4

Explore (EDA)

Charts and summaries. Price is right-skewed; newer and automatic cars cost more.

5

Engineer features

Create better inputs: Brand from Name, car age from Year, log of price.

Reality check Steps 2–5 often take about 80% of a data scientist's time. Good data beats a fancy model.

THE PIPELINE · STEPS 6–10

From data to a model in use

1

Split

Train, validation and test sets, BEFORE any fitting. Cars4U: 60 / 20 / 20.

2

Train

Fit the model on the training set only. Linear regression learns one weight per feature.

3

Evaluate

Score on the validation set with the chosen metric: RMSE, F1 and so on.

4

Tune

Try other models, features and settings; keep what improves validation scores.

5

Deploy & monitor

Test once on the test set, release, then watch for drift as the market changes.

It is a loop, not a line Poor evaluation results send you back to cleaning, features or data collection.

STEP 3 IN CODE · CLEANING

Turning text into numbers the model can use

```
In [ ]:
df = pd.read_csv("used_cars_data.csv")

# "126.2 bhp" -> 126.2   ("null bhp" -> NaN)
df["power"] = pd.to_numeric(df["Power"].str.split(" ").str[0],
                            errors="coerce")
df["engine"] = pd.to_numeric(df["Engine"].str.split(" ").str[0],
                             errors="coerce")
df["Brand"] = df["Name"].str.split(" ").str[0].str.lower()
df = df.dropna(subset=["Price"])       # no label = cannot train
print(df.shape)
```

```
Out:
(6019, 17)
```

Units in text models need numbers, so strip 'bhp' and 'CC'
errors='coerce' bad values become NaN instead of crashing
Brand 2,041 car names → 32 brands

1,234 rows had no price: they cannot teach a supervised model, so they are set aside.

TRAIN · VALIDATE · TEST

Three sets, three different jobs



Training set = homework

3,505 cars. The model studies these and adjusts its weights to fit them. It sees the answers.

Validation set = mock exam

1,169 cars. We compare models and settings here, and pick the best. Used many times while we tune.

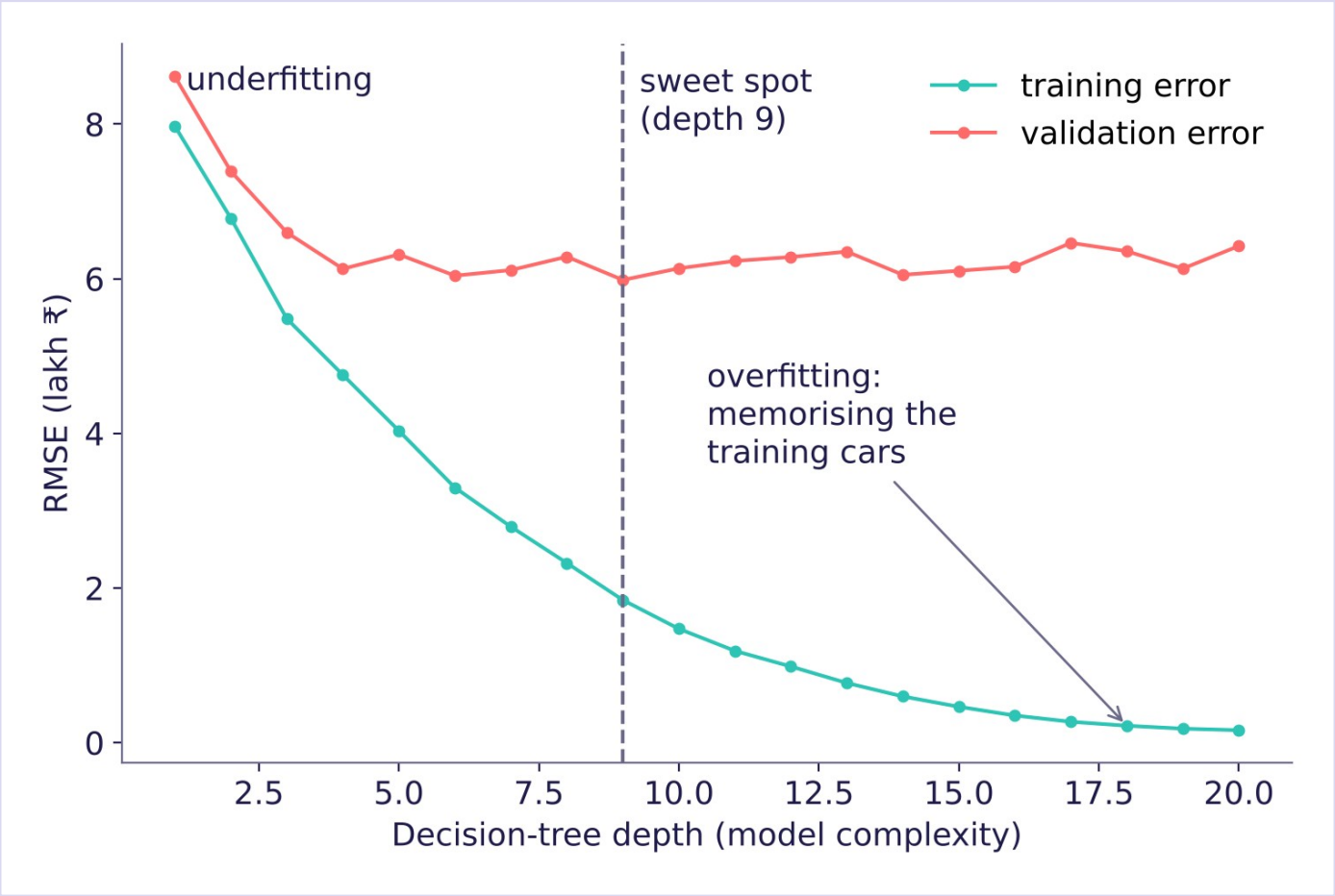
Test set = final exam

1,169 cars. Locked away until the very end, and used once for an honest estimate of real-world performance.

Golden rule never make decisions using the test set. If you tune on it, it stops being a fair exam.

OVERFITTING · REAL CURVE

When the model memorises instead of learning



Read the curve

- Depth 1: training RMSE 7.97, validation 8.62. Too simple: underfitting
- Depth 9: validation is lowest (5.98). The sweet spot
- Depth 20: training RMSE 0.16: almost perfect! But validation is 6.42
- The gap between the two lines is overfitting: great on homework, worse in the exam

FIT IN THREE PICTURES

Underfitting, good fit, overfitting

Underfitting

Too simple to capture the pattern. High error on training AND validation. Like a student who skimmed one chapter. Fix: a richer model, better features.

Good fit

Learns the real pattern, not the noise. Training and validation errors are both low and close together. This is the goal.

Overfitting

Memorises the training data, including its noise. Very low training error, high validation error. Like memorising past papers word for word. Fix: more data, a simpler model, regularisation, early stopping.

DATA LEAKAGE

When the answer sneaks into the inputs

What it is

Information the model would NOT have at prediction time leaks into training, so test scores look amazing and real-world results collapse.

Common causes

A feature built from the label · scaling or filling gaps using ALL data before splitting · duplicate rows in train and test · using future data to predict the past.

Cars4U demo

We added log(price) as a feature by mistake. Test R^2 jumped from 0.85 to 0.9999. Too good to be true, because the answer was in the question.

Rule of thumb split first, then clean and scale using the training set only. If a score looks too good, suspect leakage.

SPOT THE LEAK · 4 MIN

Leakage or not? Explain why.

1. Predicting hospital readmission, using 'discharge medication given at readmission'
2. Scaling all 6,019 cars with StandardScaler, then splitting train and test
3. Predicting car price using the car's age and kilometres driven
4. Predicting a student's final grade using the 'grade letter' column

SUGGESTED ANSWERS

Spot the leak: suggested answers

1

Readmission medication

Leak. That information only exists AFTER readmission, so it is future information.

2

Scaling before splitting

Leak (a subtle one). The mean and spread include the test cars. Fit the scaler on training data only.

3

Age and kilometres

Safe. Both are known when the car is listed, before the price is set.

4

Grade letter

Leak. The letter is just the final grade in disguise: it IS the label.

4

PART FOUR · EVALUATION

How good is the model, really?

Confusion matrix, classification metrics and regression metrics.

Metrics
worked by hand, then on 1,169 real test cars

THE CONFUSION MATRIX

Four ways a yes/no prediction can land

ACTUAL \ PREDICTED	Spam	Not spam
Spam	**TP · True positive** Said spam, and it was spam. Caught it!	**FN · False negative** Said not spam, but it was spam. Missed it
Not spam	**FP · False positive** Said spam, but it was a real email. False alarm	**TN · True negative** Said not spam, and it was fine. Correctly ignored

How to read the names

- Second word = what the model SAID (positive / negative)
- First word = was it right? (true / false)
- Positive is just 'the class we care about', not 'good'
- Errors: FP = false alarm (Type I), FN = miss (Type II)

WORKED EXAMPLE · BY HAND

Number-plate detector: 10 test images

Image	Actual	Predicted	Result	Why
1	Plate	Plate	TP	Found a real plate
2	Plate	No plate	FN	Missed a real plate
3	Plate	Plate	TP	Found a real plate
4	No plate	Plate	FP	False alarm
5	No plate	No plate	TN	Correctly ignored
6	Plate	Plate	TP	Found a real plate
7	No plate	No plate	TN	Correctly ignored
8	Plate	No plate	FN	Missed a real plate
9	No plate	Plate	FP	False alarm
10	Plate	Plate	TP	Found a real plate

WORKED EXAMPLE · BY HAND

Count them into the matrix

ACTUAL \ PREDICTED	Plate	No plate
Plate	**TP · True positive** **4** images 1, 3, 6, 10	**FN · False negative** **2** images 2, 8
No plate	**FP · False positive** **2** images 4, 9	**TN · True negative** **2** images 5, 7

Total = 10

Check your counts

- Actual plates = TP + FN = 4 + 2 = 6
- Actual no-plates = FP + TN = 2 + 2 = 4
- Predicted plates = TP + FP = 6
- All four boxes add up to 10 images

FIVE METRICS · BY HAND

Every metric from the same four numbers

Accuracy	`(TP + TN) / total`	Of ALL predictions, how many were right?	**(4+2)/10 = 60%**
Precision	`TP / (TP + FP)`	When it says 'plate', how often is it right? (few false alarms)	**4/6 = 66.7%**
Recall	`TP / (TP + FN)`	Of the real plates, how many did it find? (few misses). Also called sensitivity	**4/6 = 66.7%**
F1-score	`2 × P × R / (P + R)`	One number that balances precision and recall (harmonic mean)	**66.7%**
Specificity	`TN / (TN + FP)`	Of the real no-plates, how many did it correctly reject?	**2/4 = 50%**

Notice 60% accuracy hides the detail: the model is decent at finding plates but raises false alarms on half of the empty images.

REAL MODEL · CARS4U

Will this car sell for more than ₹10 lakh?

ACTUAL \ PREDICTED	> ₹10 L	≤ ₹10 L
> ₹10 L	TP · True positive **240** pricey cars correctly flagged	FN · False negative **49** pricey cars missed
≤ ₹10 L	FP · False positive **33** cheaper cars wrongly flagged	TN · True negative **847** cheaper cars correctly passed

Total = 1169

The set-up

- Logistic regression on 8 features: age, km, power, engine, mileage, seats, automatic, diesel
- Tested on 1,169 unseen cars
- Only 24.8% of cars are over ₹10 lakh, so the classes are imbalanced
- sklearn prints this as [[TN FP] [FN TP]]: check the order!

REAL MODEL · CARS4U

The five metrics on 1,169 real test cars

Accuracy	`(TP + TN) / total`	Right on most cars overall	**1087/1169 = 93.0%**
Precision	`TP / (TP + FP)`	When it says 'over ₹10 L', it is usually right	**240/273 = 87.9%**
Recall	`TP / (TP + FN)`	But it misses about 1 in 6 expensive cars	**240/289 = 83.0%**
F1-score	`2 × P × R / (P + R)`	Balanced score for the 'over ₹10 L' class	**85.4%**
Specificity	`TN / (TN + FP)`	Cheaper cars are almost always passed correctly	**847/880 = 96.3%**

Business reading a dealer using this would rarely overprice (high precision) but would undersell 49 pricey cars (the FNs).

THE ACCURACY TRAP

Why 75% accuracy can mean a useless model

The lazy model

Ignore the features and always answer '≤ ₹10 lakh'. Because 75.2% of cars are cheaper, it scores 75.3% accuracy on our test set.

But look closer

Recall for expensive cars = 0%. It never finds a single one. Precision cannot even be computed. Accuracy hid all of that.

Worse in medicine

If 1% of patients have a disease, 'everyone is healthy' is 99% accurate and misses every sick patient.

Lesson with imbalanced classes, always report precision, recall and F1, and compare against a 'lazy' baseline.

CHOOSING A METRIC

Which metric matters most here?

Situation	Focus on	Because…
Cancer or COVID screening	Recall	A missed sick patient (FN) is far worse than a false alarm that leads to a second test
Spam filter	Precision	A real email lost in spam (FP) is worse than seeing one junk email
Fraud detection	Recall + F1	Missing fraud is costly, but too many false alarms annoy customers
Airport security screening	Recall	Better to check a bag twice than miss a threat
Balanced classes, equal costs	Accuracy	When errors cost the same and classes are even, accuracy is fine
Confirming who is healthy	Specificity	Correctly clearing healthy people avoids needless treatment

PRECISION OR RECALL? · 4 MIN

Which matters more, and why?

1. A model that flags students at risk of failing so tutors can help early
2. A YouTube filter that removes videos it thinks are copyright violations
3. A smoke alarm in a student hostel
4. A model choosing which customers get a free AED 500 voucher

SUGGESTED ANSWERS

Precision or recall: suggested answers

1

At-risk students

Recall. Missing a struggling student (FN) costs more than an extra check-in with a student who is fine (FP).

2

Copyright filter

Precision. Wrongly removing a creator's original video (FP) is harmful and unfair.

3

Smoke alarm

Recall. A missed fire (FN) is catastrophic; a false alarm is only annoying.

4

Free voucher

Precision. Every voucher given to the wrong customer (FP) costs AED 500.

REGRESSION METRICS

MAE, MSE, RMSE: how far off are the numbers?

MAE	`mean( |actual − predicted| )`	Average size of the error, in the label's units. Easy to explain: 'off by ₹X lakh on average'	**units: lakh ₹**
MSE	`mean( (actual − predicted)² )`	Squares each error first, so big mistakes count much more. Used as the training loss	**units: lakh ₹²**
RMSE	`√MSE`	Back in the label's units, but still punishes big errors. Always ≥ MAE	**units: lakh ₹**

Lower is better for all three and 0 means perfect. If RMSE is much bigger than MAE, a few predictions are badly wrong.

WORKED EXAMPLE · 5 REAL TEST CARS

Calculating MAE, MSE and RMSE by hand

Car (test set)	Actual (₹L)	Predicted (₹L)	Error	|Error|	Error²
BMW X5 3.0d	17.8	25.6	+7.8	7.8	60.84
Volvo S60 D5	17.6	27.2	+9.6	9.6	92.16
Hyundai Verna VTVT	11.9	14.8	+2.9	2.9	8.41
Toyota Innova 2.5	9.5	7.8	-1.7	1.7	2.89
Audi Q3 35	25.7	24.7	-1.0	1.0	1.00
Total				23.0	165.30

MAE = 23.0 / 5 = 4.60 **MSE** = 165.30 / 5 = 33.06 **RMSE** = √33.06 = 5.75 lakh ₹

REAL MODEL · ALL 1,169 TEST CARS

How the regression model did overall



Test-set scores

- MAE = 3.52 lakh ₹: off by about ₹3.5 lakh on average
- MSE = 29.14 RMSE = 5.40 lakh ₹
- R^2 = 0.75: explains about three-quarters of the price variation
- Dots far above the line are cheaper cars it overpriced; the spread grows for expensive cars
- Next step: add Brand, use log(price), or try a tree model

METRICS IN CODE

scikit-learn does the arithmetic

```
In [ ]:
from sklearn.metrics import (mean_absolute_error,
        mean_squared_error, r2_score, confusion_matrix)

pred = reg.predict(X_test)
mae  = mean_absolute_error(y_test, pred)
rmse = mean_squared_error(y_test, pred) ** 0.5
print(f"MAE {mae:.2f}  RMSE {rmse:.2f}  R2 {r2_score(y_test, pred):.2f}")

print(confusion_matrix(yc_test, clf.predict(Xc_test)))
```

```
Out:
MAE 3.52  RMSE 5.40  R2 0.75
[[847 33]
 [ 49 240]]
```

Order! sklearn prints [[TN FP] [FN TP]]
classification_report gives precision, recall and F1 for every class in one call

Calculate by hand first, then let the code check you: the numbers must match.

MAE OR RMSE?

Picking the right regression metric

Use MAE when…

You want an easy, honest 'average miss' to tell a manager, and an occasional big error is no worse than several small ones.

Use RMSE when…

Big errors are especially bad. Mispricing a ₹40 lakh car by ₹15 lakh hurts much more than three ₹5 lakh misses.

Use R^2 when…

You want a unit-free score from 0 to 1 to compare models on the same data: 'how much of the variation do we explain?'

Report more than one Cars4U: MAE 3.52 with RMSE 5.40 tells us most cars are close, but a few are far off.

EXIT PROBLEM · 10 MIN

Calculate it yourself

1. An 'at-risk student' model on 200 students: TP = 30, FP = 10, FN = 20, TN = 140
2. (a) Accuracy (b) Precision (c) Recall (d) F1 (e) Specificity
3. Is this model good enough for tutors to rely on? Use one metric to argue.
4. Regression: actual [10, 12], predicted [9, 15]. Find MAE, MSE and RMSE.

SUGGESTED ANSWERS

Exit problem: worked answers

1

(a) Accuracy and (b) Precision

Accuracy = (30 + 140) / 200 = 85%. Precision = 30 / (30 + 10) = 75%.

2

(c) Recall and (d) F1

Recall = 30 / (30 + 20) = 60%. F1 = 2 × 0.75 × 0.60 / 1.35 = 66.7%.

3

(e) Specificity and the verdict

Specificity = 140 / 150 = 93.3%. Not yet: recall is only 60%, so 20 of 50 struggling students are missed. 85% accuracy hides that.

4

Regression

Errors 1 and 3. MAE = (1 + 3) / 2 = 2. MSE = (1 + 9) / 2 = 5. RMSE = $\sqrt{5} \approx 2.24$.

Learn from examples.

Measure honestly.

Next: try these metrics on your own Cars4U model. Change one feature, re-split, and see which metric moves.

